MSc in Computer Science and Engineering

Planning, Learning and Decision Making

Lab 3: Fishing Game
Reinforcement Learning

Introduction

In previous labs, you modeled environments with known transition probabilities. In this lab, you will work with a simulated environment: the **Fishing Game**. The goal is to control a green "fishing bar" to keep it behind a moving fish. The environment dynamics are complex (involving velocity, acceleration, and random fish movement), making it suitable for Reinforcement Learning (RL) where the agent must learn from interaction.

The MDP Environment

The environment consists of a continuous state space and a discrete action space.

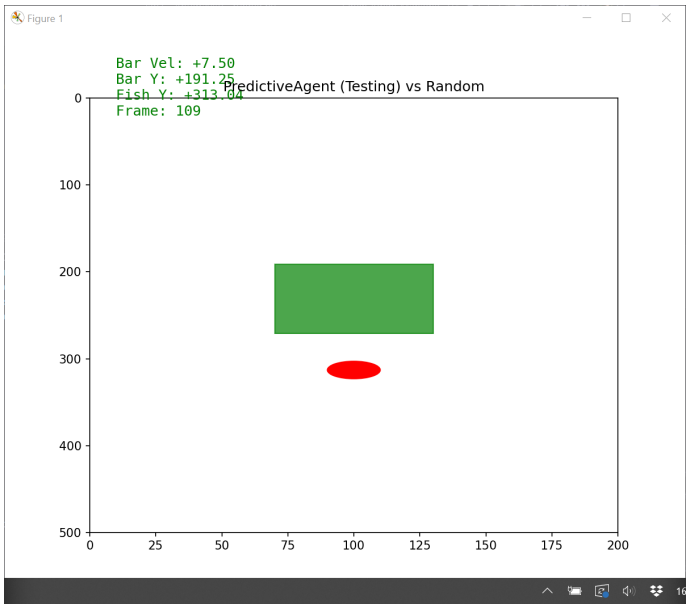

Figure 1: Fishing Game. The goal is to grab the fish. The fish is represented by a red circle and it moves randomly (different fish have different dynamics). The green bar is the bait that needs to be keep over the fish for some time. You can reel in (making the bait go up) or leave it (allowing the bait to go down due to gravity. You'll have access to the bait and fish position and bait velocity.

State Space

The state s_t is defined by four factors:

1. **Fish Position (y_{fish})**: Vertical position of the fish [0, 500].
2. **Bar Position (y_{bar})**: Vertical position of the fishing bar [0, 500].
3. **Bar Velocity (v_{bar})**: Current velocity of the bar [-17, +11.5].

Since the state space is continuous, you will need to apply **discretization** (e.g., binning) to use tabular RL methods.

Action Space

The agent can choose between two discrete actions at each step:

- **Action 0 (Nothing/Gravity):** The bar falls due to gravity ($v_{t+1} = v_t + \text{Gravity}$).
- **Action 1 (Thrust):** The bar is pushed upwards ($v_{t+1} = v_t - \text{Thrust}$).

Cost Function $C(s, a)$

The goal is to keep the fish inside the green bar:

$$C_t = \begin{cases} -1 & \text{if fish is inside the bar} \\ +1 & \text{otherwise} \end{cases} \tag{1}$$

Episode Termination

The two conditions that terminate an episode are:

1. **Win**: the cumulative time that the fish is inside the green bar surpasses the threshold T_{win};
2. **Lose**: the cumulative time that the fish is outside the green bar surpasses the threshold T_{lose}.

Code

There are 4 files to train and test the agents.

- **agents.py** - main agent class and the 3 derived classes where you implement the RL algorithms.
- **runner.py** - this is the main loop to train and test, you can change it if you want to make more extensive logging of the results.

- **fishing_logic.py** - this is the game dynamics, **do not change**. The visualization should be enough to understand the dynamics of the problem. If needed you can review it to better understand the environment dynamics. But as a real agent, usually you do not have access to this information.

- **visualize.py** - just for the visualization, **do not change**.

Criteria

As in all engineering problems each solution has advantages or disadvantages. All questions allow different answers.

When selecting one approach you should explain the advantages/disadvantages and provide results that show them. There is no time for a detailed analysis so just provide simple evidence. You do not need to choose the best solution, as in most cases there is no best solution, just discuss the compromises between approaches.

1 Questions

1.1 Q-Learning vs SARSA

Implement **Q-Learning** (off-policy) and **SARSA** (on-policy) for the Fishing Game. You must implement the following code snippets:

1. Since the simulator provides continuous values, but tabular RL requires discrete states, start by selecting a state representation, and explain your discretization strategy. Implement the `TDAgent.discretize_state` method inside **agents.py** file.

2. Implement the $\epsilon - greedy$ exploration strategy, that will be used by the learning agents. Here, modify the `TDAgent.get_action` method at **agents.py** file.

3. Implement Q-Learning and Sarsa. Populate the learning methods in each agent's class: `QLearningAgent.learn` and `SarsaLearningAgent.learn`, both located inside **agents.py** file.

Compare the results obtained by training and testing all agents: *Predictive Agent* (already implemented), *Q-Learning*, and *SARSA*.

- Train the agents for at least 2500 episodes

- Compare the cumulative cost obtained during training.

- Compare the performance obtained at test time, using the metrics already provided: cumulative costs, win-rate, avg. costs, and avg. steps.
- Discuss the differences in the policies learned and the convergence rate.

1.2 Convergence Analysis

Now, consider just the algorithm that worked better. Analyze the convergence of your best agent:

1. Run the training multiple times with different random seeds. Plot the mean and standard deviation of the training and testing cumulative costs. Detail and explain the robustness of the training runs (e.g., their consistency around the mean).
2. Is the final policy optimal? How do you know? Explain.
3. What is the impact of the exploration parameters? Run at least two more experiments where you change:
   (a) `epsilon` parameter;
   (b) `epsilon_decay` parameter.

   Compare and interpret the obtained results against the original configuration. Use the following metrics: train cumulative costs, test cumulative costs, and avg. steps.
4. If you force the training in a single fish type, e.g., `FishingGameLogic(fish_name="Sturgeon")`, will it work on others? Compare and interpret the obtained results against the original configuration. Use the following metrics: train cumulative costs, test cumulative costs, and avg. steps. You need to change the **runner.py** to do this.

1.3 Better Policies

Do you have ideas to improve the learning agent? Change the state? Other representations? Suggest and implement one of them.

You probably will not have time to tune it to improve the results, just discuss your ideas and the results you obtain.